PRESEARCH.md

Complete before writing any code

This document captures the design decisions and architectural reasoning for FleetGraph, a project intelligence agent for Ship. The goal is to make informed decisions about the agent's responsibilities and architecture before implementation.

Phase 1: Define Your Agent

1. Agent Responsibility Scoping

What events in Ship should the agent monitor proactively?

Ship's unified document model and issue lifecycle provide clear signals for proactive monitoring:

- **Stale issues**: Issues stuck in `todo` or `in_progress` for >3 days with no `document_history` updates
- **Sprint health**: Active sprints where completion percentage is trending below target relative to days remaining, or where `confidence` scores are dropping
- **Missing standups**: Team members (person documents) with no standup document created for the current day by a threshold time
- **Triage queue aging**: Issues in `triage` state >24 hours without being accepted or rejected
- **Overdue items**: Issues with `due_date` in the past that are not in `done` or `cancelled` state
- **Workload imbalance**: Disproportionate `assignee_id` distribution across active issues relative to person `capacity_hours`
- **Unassigned work**: Issues in active sprints with no `assignee_id`

All of these can be detected via Ship's existing REST API endpoints (`GET /issues` , `GET /weeks` , `GET /standups/status` , `GET /team/grid`).

What constitutes a condition worth surfacing?

Decision: **Moderate alerting** - surface clear problems AND trending-negative risk signals. Specifically:

- **Always surface**: Issues stale >3 days, missed standups >2 consecutive days, sprints with <50% completion in final 2 days, overdue items, triage items aging >24h
- **Surface as risk signals**: Confidence scores dropping across consecutive polls, velocity slowing vs. previous sprint, unreviewed items piling up (>5 in `in_review`), workload imbalance >2x standard deviation
- **Do NOT surface**: Minor fluctuations, single missed standups, items recently moved to new states (grace period of 4 hours after state change)

The agent stays quiet unless something is clearly wrong or measurably trending negative.

What is the agent allowed to do without human approval?

Decision: **Read-only + notify**. The agent can autonomously:

- Query all Ship API endpoints to gather data
- Analyze patterns, trends, and relationships across documents
- Generate insights and findings
- Deliver notifications/alerts to the agent findings panel
- Cache data for graceful degradation

The agent NEVER modifies Ship data on its own. All write actions require explicit human confirmation.

What must always require confirmation?

Decision: **All write actions**. Any operation that changes Ship state requires human approval:

- Issue state changes (move, close, cancel, reopen)
- Assignment changes (reassign, unassign)
- Priority or property updates
- Creating new documents (issues, comments, standups)
- Bulk operations (archive, delete, restore)
- Sprint scope changes (add/remove issues from sprint)

The agent proposes actions with rationale; the human clicks confirm or dismiss.

How does the agent know who is on a project?

The agent queries multiple Ship API endpoints to build a project membership graph:

1. `GET /workspaces/:id/members` - All workspace members with roles (`admin` / `member`)
2. Person documents (`GET /documents?type=person`) - Editable profiles linked to users via `properties.user_id`
3. Project/program ownership - `properties.owner_id` and `properties.accountable_id` on project and program documents
4. Issue assignments - `properties.assignee_id` on issue documents
5. `GET /team/grid` - Full team allocation view showing who is assigned where
6. `document_associations` - Links issues to projects, programs, and sprints

The person-to-project mapping is derived by traversing: person -> assigned issues -> associated projects/programs/sprints.

How does the agent know who to notify?

Notification targets are determined by role and document relationships:

- **Issue owner/assignee**: `properties.assignee_id` on the issue
- **Project owner**: `properties.owner_id` on the project document
- **Sprint owner**: `properties.owner_id` on the week document (person accountable for that sprint)
- **Program accountable**: `properties.accountable_id` on the program document (RACI approver)
- **Manager chain**: `properties.reports_to` on person documents for escalation
- **Workspace admins**: `workspace_memberships` with `role = 'admin'` for org-wide alerts

The agent delivers findings to the dedicated agent findings panel, targeted to the relevant person(s) based on these relationships.

How does the on-demand mode use context from the current view?

The chat interface is embedded in Ship's UI, scoped to the current document. When a user opens the chat:

1. The **context node** receives the current `document_id` and `document_type` from the frontend
2. It fetches that document via `GET /documents/:id` including all properties
3. It fetches associations via `GET /documents/:id/associations` (parent project, sprint, program)

4. It fetches related data: assignee's other issues, sprint sibling issues, document history via `GET /issues/:id/history`
5. It fetches backlinks via `GET /documents/:id/backlinks` for cross-references

This context is injected into the graph state before the reasoning node runs, so the LLM's analysis is grounded in the specific document the user is viewing. A chat on an issue knows that issue's full context; a chat on a sprint knows all sprint issues and their states.

---

2. Use Case Discovery

Seven use cases covering Director, PM, and Engineer roles across proactive and on-demand modes:

#	Role	Trigger	Agent Detects / Produces	Human Decides
1	PM	Proactive poll (every 2-3 min for active sprints)	**Stale Issue Detection**: Issues stuck in `todo`/`in_progress` >3 days with no history updates. Surfaces issue title, assignee, days stale, and last activity.	Whether to reassign, reprioritize, ping the assignee, or dismiss
2	PM / Director	Proactive poll (every 2-3 min for active sprints)	**Sprint Health Monitor**: Active sprint completion %, confidence trend, remaining days, at-risk issues. Flags sprints below expected velocity curve.	Whether to adjust scope, extend sprint, reassign work, or accept the risk
3	Engineer / PM	Proactive daily check (at configurable threshold time)	**Missing Standup Alerts**: Team members without standup documents for the current day. Lists who is missing and their last standup date.	Whether to follow up with the person, create a standup on their behalf, or dismiss
4	PM	Proactive poll (every 5 min)	**Triage Queue Aging**: Issues in `triage` state >24h without accept/reject action. Shows queue depth, oldest items, and suggested priority based on source/content.	Whether to accept (move to backlog), reject with reason, or delegate triage to another PM
5	Director / PM	Proactive weekly analysis	**Workload Imbalance Detection**: Compares `assignee_id` distribution across active issues against person `capacity_hours`. Flags overloaded (>150% capacity) or idle (<25% capacity) team members.	Whether to rebalance assignments, adjust capacity settings, or acknowledge the imbalance
6	PM / Director	On-demand (chat on sprint view)	**On-Demand Sprint Summary**: Fetches all sprint issues, groups by state, calculates velocity vs. plan, identifies blockers and at-risk items. Produces narrative summary with key metrics.	What follow-up actions to take, which items to escalate, whether to adjust goals

7	Engineer	On-demand (chat on issue view)	**On-Demand Issue Context**: Fetches issue history, parent project status, sibling issues in sprint, blocker chains, due date risk, and assignee workload. Produces actionable context brief.	How to prioritize this issue relative to other work, whether to flag blockers, when to start

---

3. Trigger Model Decision

Decision: Adaptive polling with smart intervals.

Ship has no native webhook or outbound event system. The existing infrastructure supports pull-based data access via REST API, plus internal WebSocket broadcasts for real-time collaboration. Given this constraint, adaptive polling is the most practical and cost-efficient approach.

Polling intervals by context:

Context	Interval	Rationale
Active sprints (status = 'active')	Every 2-3 minutes	Meets <5 min detection latency requirement. Active work changes frequently.
Sprint planning phase	Every 5 minutes	Changes are less time-sensitive during planning
Completed/inactive sprints	Every 30 minutes	Minimal changes expected; only check for retroactive updates
Standup checks	Once daily at configurable threshold (e.g., 10:30 AM)	Standups are a daily ritual; checking more often is wasteful
Workload analysis	Once daily or on-demand	Doesn't change rapidly; expensive to compute
Triage queue	Every 5 minutes during business hours	Triage is a PM workflow that happens during work hours

Tradeoffs considered:

Approach	Pros	Cons
Uniform polling	Simple to implement	Wasteful API calls on inactive resources; doesn't scale
Webhook-based	Near-instant detection	Requires building webhook dispatch into Ship API (significant new work)
Adaptive polling (chosen)	60-70% fewer API calls vs. uniform; meets latency target; works with existing API	More complex scheduling logic; slight detection delay on low-priority contexts

Hybrid (poll + WebSocket)	Lowest latency for active documents	WebSocket connection management is complex; Ship's WS is designed for document collab, not event streaming

Scaling projections:

- **At 100 projects**: ~500 API calls/hour (assuming 20% active at any time). Negligible load.
- **At 1,000 projects**: ~5,000 API calls/hour. Manageable with connection pooling and request batching.
- **At 10,000 projects**: ~50,000 API calls/hour. Would need request batching, response caching, and potentially a dedicated read replica or event stream from Ship.

Staleness tolerance: For the "moderate" alert level, data up to 5 minutes old is acceptable for all use cases. The only time-critical detection is active sprint issues, which get the 2-3 minute interval.

Phase 2: Graph Architecture

4. Node Design

Context Nodes:

- `resolve_context` - Determines trigger type (proactive vs. on-demand), current user (if on-demand), current document context (document_id, document_type), and workspace_id
- `resolve_user_role` - Fetches the invoking user's workspace membership role and person document properties

Fetch Nodes (run in parallel where possible):

- `fetch_issues` - `GET /issues` with relevant filters (state, sprint, assignee)
- `fetch_sprint` - `GET /weeks/:id` + `GET /weeks/:id/issues` for active sprint data
- `fetch_team` - `GET /team/grid` for team allocation
- `fetch_standups` - `GET /standups/status` for standup completion
- `fetch_history` - `GET /issues/:id/history` for change timeline (on-demand mode)
- `fetch_associations` - `GET /documents/:id/associations` for relationship graph (on-demand mode)

Parallel groupings:

- Proactive run: `fetch_issues` || `fetch_sprint` || `fetch_team` || `fetch_standups` (all in parallel)
- On-demand issue context: `fetch_history` || `fetch_associations` || `fetch_sprint` (all in parallel)

Reasoning Nodes:

- `analyze_health` - LLM reasons about sprint health, issue staleness, workload balance. Produces structured findings with severity (info/warning/critical)
- `analyze_context` - LLM reasons about on-demand query in context of fetched data. Produces narrative analysis and suggested actions

Conditional Edges:

- After `analyze_health`: routes to `action_propose` if findings detected, routes to `log_clean_run` if no issues found

- After `analyze_context`: routes to `action_propose` if actionable suggestions exist, routes to `respond_info_only` if query is purely informational
- Severity-based branching: critical findings route through `escalation_check` node; warning/info findings go directly to output

Action Nodes:

- `action_propose` - Formats findings into proposed actions with confirm/dismiss/snooze options
- `escalation_check` - For critical findings, determines if escalation to manager chain is warranted
- `respond_info_only` - Formats informational response for on-demand queries with no action needed

Human-in-the-Loop Gates:

- `confirmation_gate` - Pauses graph execution, surfaces proposed action to user in agent findings panel, waits for confirm/dismiss/snooze response
- On confirm: routes to `execute_action` node
- On dismiss: routes to `log_dismissed` node (records finding ID to prevent re-surfacing)
- On snooze: routes to `schedule_resurface` node (queues finding for later)

Error and Fallback Nodes:

- `api_error_handler` - Catches Ship API failures, retries 2x with exponential backoff, falls back to cached data
- `llm_error_handler` - Catches Claude API failures, retries once, falls back to rule-based analysis (threshold checks without LLM reasoning)
- `graceful_degrade` - Marks findings as stale-data-based when operating on cached data

5. State Management

State carried across a single graph session:

- `context`: trigger type, user info, document context, workspace_id
- `fetched_data`: all API responses from fetch nodes
- `findings`: structured list of detected issues/risks from reasoning nodes
- `proposed_actions`: actions awaiting human confirmation
- `errors`: any API failures encountered during the run

State persisted between proactive runs (stored in lightweight datastore - Redis or SQLite on Railway):

- `last_poll_timestamps`: per-project/sprint timestamps of last successful poll
- `last_known_updated_at`: per-document `updated_at` values to detect changes
- `dismissed_findings`: set of finding IDs that were permanently dismissed
- `snoozed_findings`: finding IDs with resurface timestamps
- `cached_responses`: last successful API response per endpoint (for fallback)
- `alert_history`: recent findings to prevent duplicate alerts within a time window

Avoiding redundant API calls:

- Only fetch documents with `updated_at > last_known_updated_at` (use query params where supported)
- Cache team grid data (changes infrequently) with 15-minute TTL
- Skip polling for sprints in `completed` status unless explicitly checked

- Deduplicate findings against `alert_history` before surfacing

6. Human-in-the-Loop Design

Which actions require confirmation? All write actions, per the autonomous scope decision. Specifically:

- Moving an issue to a different state
- Reassigning an issue to a different person
- Changing issue priority
- Adding/removing issues from a sprint
- Creating new documents or comments
- Any bulk operation

What does the confirmation experience look like in Ship? A dedicated **Agent Findings** panel in Ship's UI that accumulates proposals:

- Each finding is a card with: severity badge (info/warning/critical), title, description, affected document link, proposed action button(s), dismiss button, snooze button
- Cards are sorted by severity (critical first) then recency
- Confirm triggers the write action via Ship API
- Panel shows a count badge on its icon when new findings arrive
- Panel is accessible from any view in Ship (persistent sidebar tab or icon rail item)

What happens if the human dismisses or snoozes?

- **Dismiss**: Finding is permanently suppressed. Its ID is stored in `dismissed_findings`. The agent will not re-surface this exact finding even if the condition persists.
- **Snooze**: User selects an interval (1 hour / 4 hours / next day). Finding is stored in `snoozed_findings` with a resurface timestamp. When the timer expires, the finding re-enters the panel if the underlying condition still exists (re-evaluated on next poll).
- **Stale auto-dismiss**: If the underlying condition resolves before a snoozed finding resurfaces (e.g., the stale issue gets updated), the finding is auto-dismissed and does not reappear.

7. Error and Failure Handling

What does the agent do when Ship API is down? Decision: **Retry + cached fallback**.

1. On API failure: retry 2x with exponential backoff (1s, then 3s delay)
2. If still failing: use last-known cached response from `cached_responses` store
3. Mark any findings produced from cached data with a staleness indicator: "Based on data from X minutes ago"
4. Skip all write action proposals when operating in degraded mode (read-only analysis only)
5. Log the failure with timestamp, endpoint, error code for observability

How does it degrade gracefully?

- **Partial failure**: If some fetch nodes fail but others succeed, the agent runs reasoning on available data and notes which data sources were unavailable
- **Full API outage**: Agent enters standby mode, logs the outage, and retries on next poll interval. No findings surfaced from fully stale data.
- **LLM failure**: Falls back to rule-based threshold checks (e.g., "issue stale >3 days" doesn't need LLM reasoning). Findings from rule-based fallback are marked as "simplified analysis"

What gets cached and for how long?

- Issue list responses: cached with 5-minute TTL (matches poll interval)
- Sprint data: cached with 5-minute TTL
- Team grid: cached with 15-minute TTL (changes infrequently)
- Person documents: cached with 30-minute TTL (rarely change)
- Standup status: cached with 10-minute TTL
- Document history: not cached (only fetched on-demand)

Phase 3: Stack and Deployment

8. Deployment Model

Where does the proactive agent run when no user is present? As a **separate Railway service** (Node.js worker process) within the same Railway project as the Ship API. This provides:

- Independent scaling from the Ship API
- Own resource allocation (CPU/memory)
- Independent restart/deploy lifecycle
- Railway's built-in health checks and auto-restart

How is it kept alive?

- Railway's process management handles restarts on crash
- The agent runs an internal health check endpoint that Railway monitors
- The adaptive polling scheduler runs as a persistent event loop (node-cron or custom scheduler)
- Graceful shutdown handling for Railway's SIGTERM during deploys

How does it authenticate with Ship without a user session? Via Ship's **API token system** (`api_tokens` table). A long-lived API token is created for the FleetGraph service account:

- Bearer token authentication (skips CSRF protection)
- Rate-limited per token
- Scoped to the workspace(s) the agent monitors
- Token stored as a Railway environment variable (`FLEETGRAPH_API_TOKEN`)
- No session timeout concerns (API tokens don't use the 15-minute session mechanism)

9. Performance

How does your trigger model achieve the < 5 minute detection latency goal?

- Active sprints are polled every 2-3 minutes, guaranteeing worst-case detection latency of 3 minutes for the most time-critical context
- The adaptive interval ensures that resources are focused on active work, not stale/completed sprints
- Fetch nodes run in parallel, reducing per-poll execution time to ~1-2 seconds for API calls + ~2-3 seconds for LLM reasoning = ~4-5 seconds total per poll cycle
- Total latency budget: 3 min (worst-case poll gap) + 5 sec (execution) = ~3 min 5 sec, well within the 5-minute target

What is your token budget per invocation?

- **~4K tokens per run**: ~2K input (fetched Ship data context) + ~2K output (reasoning + findings)
- **Model**: Claude Sonnet 4.6 ($3/MTok input, $15/MTok output)

- **Cost per run**: ~~$0.006 input + ~$0.030 output = **~~$0.036 per run**

Where are the cost cliffs in your architecture?

1. **Number of active projects**: Each active project generates ~20 poll cycles/hour at 3-minute intervals. Cost scales linearly with active project count.
2. **On-demand query depth**: Complex on-demand queries (cross-project analysis) may require more context tokens, potentially 2-3x the base budget.
3. **LLM reasoning vs. rule-based**: The biggest cost item is the reasoning node. For simple threshold detections (stale >3 days), rule-based checks could bypass the LLM entirely, reducing costs by ~90% for those checks.
4. **Cache miss storms**: If Redis/cache goes down, all polls become full-fetch cycles, spiking API call volume and potentially hitting rate limits.

Cost projections at scale (Claude Sonnet 4.6 at ~$0.036/run):

Scale	Proactive runs/day	On-demand runs/day	Total runs/day	Monthly cost
100 users (~20 active projects)	~9,600 (20 projects x 20 runs/hr x 24hr)	~200 (2/user/day)	~9,800	~$10,584/mo
1,000 users (~200 active projects)	~96,000	~2,000	~98,000	~$105,840/mo
10,000 users (~2,000 active projects)	~960,000	~20,000	~980,000	~$1,058,400/mo

Note: These projections assume all runs use LLM reasoning. In practice, rule-based pre-filtering (skip LLM when no changes detected since last poll) would reduce actual LLM invocations by 70-80%, bringing costs to approximately:

- 100 users: ~$2,100-3,200/mo
- 1,000 users: ~$21,000-32,000/mo
- 10,000 users: ~$210,000-320,000/mo